

AI WEATHERWISE

Phase 7: Project Documentation

User, developer and API documentation

Project	AI WeatherWise – Deliver intelligent weather forecasts and insights
Phase	7 of 8 – Project Documentation
Technology	Node.js, Express.js, MongoDB (Mongoose), JWT, Google Gemini AI, OpenWeatherMap
Version / Date	1.0 28 September 2026

1. Project Summary

AI WeatherWise is a RESTful backend platform built with Node.js and Express.js, using MongoDB and Mongoose for data persistence. It delivers real-time weather metrics and integrates Google Gemini AI to provide intelligent summaries and personalized recommendations. The platform features secure JWT authentication, bcrypt password hashing, endpoints for managing favorite locations and fetching weather insights, a resilient fallback mode, centralized error handling and request sanitization.

2. Problem and Solution

Problem	Solution provided
Difficulty managing multiple favorite locations	Location CRUD with per-user storage and no duplicates.
No personalized outdoor or clothing advice	Gemini AI recommendations from live weather data.
Raw weather metrics are hard to interpret	Natural-language summaries and recommendations.
No secure way to sync preferences across devices	JWT-secured accounts backed by MongoDB.

3. Key Features

- User registration and login with JWT and bcrypt password hashing.
- Favorite city management (create, read, update, delete) with duplicate prevention.
- Real-time weather metrics: temperature, humidity, wind speed and condition.
- AI-generated weather summaries and recommendations using Google Gemini.
- Fallback mode that keeps the API functional without external API keys.
- Role-based route guarding, request sanitization and centralized error handling.

4. Architecture Summary

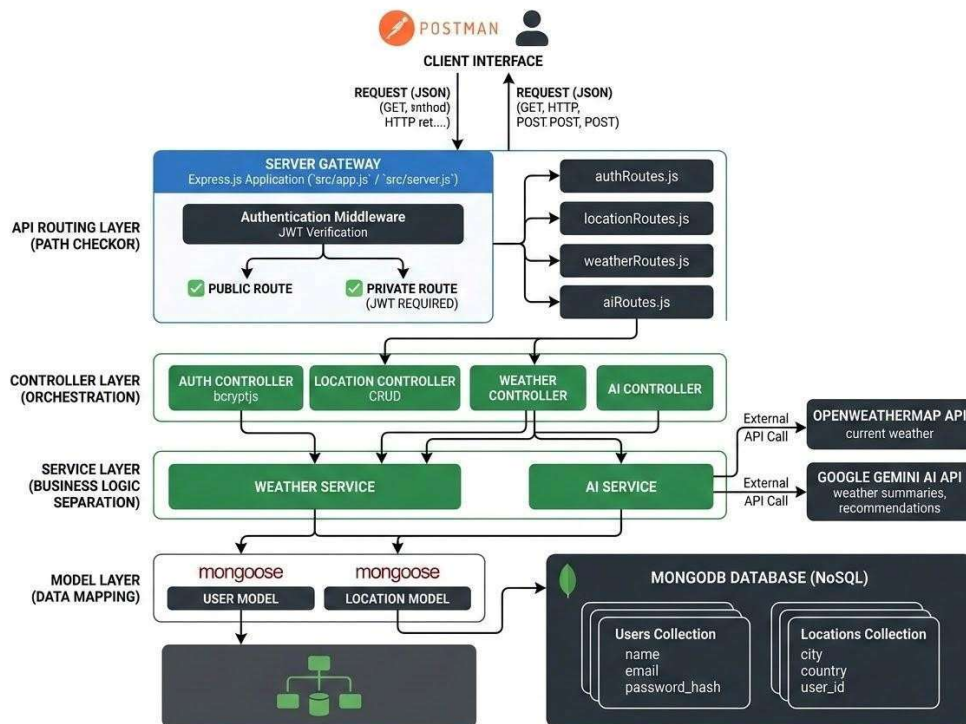


Figure 1: Layered architecture of AI WeatherWise

The project follows the MVC pattern: routes (view layer), controllers, services and Mongoose models, backed by MongoDB. External calls go to OpenWeatherMap and Google Gemini.

5. Technology Stack

Layer	Technology
Runtime	Node.js v16+
Framework	Express.js
Database	MongoDB with Mongoose ODM
Security	JWT (jsonwebtoken), bcryptjs, cors
AI	Google Gemini
Weather data	OpenWeatherMap
Tools	Postman, Visual Studio Code, Nodemon, dotenv

6. Installation and Setup Guide

6.1 Prerequisites

- Node.js v16 or above and npm v8 or above
- MongoDB running locally or a cloud connection string
- A Google Gemini API key (optional – fallback mode works without it)
- Postman for testing

6.2 Steps

1. Clone or download the repository and open it in Visual Studio Code.
2. Install dependencies:

```
npm install
```

1. Create the `.env` file in the project root (see below).
2. Start the server:

```
npm start
```

6.3 Environment variables

Variable	Description	Example
PORT	Server port	5000
MONGO_URI	MongoDB connection string	mongodb://127.0.0.1:27017/weatherwise
JWT_SECRET	Secret used to sign tokens	your_jwt_secret
GEMINI_API_KEY	Google Gemini API key	your_gemini_api_key

7. API Reference

Method	Endpoint	Auth	Description
POST	/api/auth/register	No	Create an account and receive a JWT.
POST	/api/auth/login	No	Log in and receive a JWT.
GET	/api/weather/:city	No	Current weather for a city.
POST	/api/ai/weather-recommendation	JWT	AI recommendation from weather data.
CRUD	/api/locations	JWT	Manage favorite locations.

Register – request

```
POST /api/auth/register
{
  "name": "John Doe",
  "email": "john@mail.com",
  "password": "123456"
}
```

Register – response (201 Created)

```
{
  "success": true,
  "data": { "_id": "...", "name": "John Doe", "email": "john@mail.com", "token":
"<JWT>" }
}
```

Weather – response (200 OK)

```
GET /api/weather/bangalore
{"success":true,"data":{"city":"Bangalore","temperature":34,
"humidity":79,"windSpeed":12,"condition":"Windy","isMock":true}}
```

AI recommendation – request and response

```
POST /api/ai/weather-recommendation
{ "city": "Bangalore", "temperature": 34, "humidity": 79, "condition": "Windy" }

{ "success": true, "recommendation": "Stay hydrated, and wear light cotton clothes." }
```

8. Data Models

Model	Fields
User	_id, name, email (unique), password (hashed), role (default "reader"), createdAt, updatedAt
Location	_id, user (ref User), city, country, createdAt – unique index on user + city + country

9. Roles and Access

Role	Access
Public user	Public weather endpoints without an account.
Registered user	Manages favorite locations, requests weather and AI insights, updates own profile.
Admin	Manages accounts, monitors API health and database integrity.

10. Project Structure

```
src/
  config/db.js
  controllers/  aiController.js authController.js locationController.js
  weatherController.js
  middleware/   authMiddleware.js
  models/      Location.js User.js
  routes/      aiRoutes.js authRoutes.js locationRoutes.js weatherRoutes.js
  services/
  app.js
  server.js
.env
.gitignore
package.json
postman_collection.json
README.md
```

11. Testing Summary

The register, login, weather and AI recommendation APIs were tested in Postman and returned the expected status codes (201, 200, 200, 200) with response times between 90 ms and 475 ms. Details and screenshots are in the Project Testing document.

12. Troubleshooting

Problem	Likely cause	Solution
"Database connection failed"	MongoDB not running or wrong <code>MONGO_URI</code>	Start MongoDB and check the connection string.
Port already in use	Another process uses the port	Change <code>PORT</code> in <code>.env</code> .
Invalid token	Missing or expired JWT	Log in again and send the token in the request header.
<code>isMock: true</code> in weather	No live weather API key	Add a valid key to use live data.
AI returns a generic message	Missing <code>GEMINI_API_KEY</code>	Add the key in <code>.env</code> and restart.

13. Limitations and Future Enhancements

- No graphical front end yet – the service is API-only.
- Planned entities from the ER diagram: weather history, saved AI insights, alerts, subscriptions, API usage logs and AI model registry.
- Weather alerts with severity levels and push or e-mail notifications.
- Admin dashboard for user and API-usage monitoring.
- Automated unit and integration tests, and deployment to a cloud platform.

14. Submission Links

GitHub repository (public)	<paste public repository link>
API testing video	API testing.mp4
Demo video	Backend demo.mp4
Code drive	<paste drive link>